The TCP/IP Model

what really happens when you type a URL

Watch the 3-min video

In one line: the internet runs on the 4-layer TCP/IP model (link, internet, transport, application); opening a URL costs a DNS lookup, a TCP handshake and a TLS handshake before the first byte of HTTP, which is why caching and connection reuse matter.

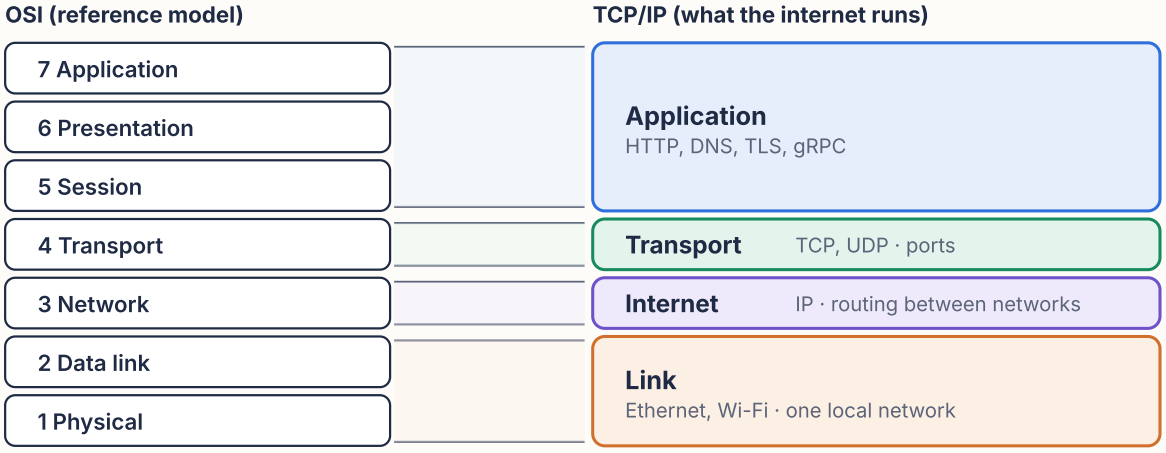
1 Four layers, not seven

The OSI model from the last lesson is a vocabulary. The internet actually runs on **TCP/IP**, a simpler four-layer model:

- **Link**: moves frames across one local network, such as Wi-Fi or Ethernet, using MAC addresses.
- **Internet**: moves packets between networks using IP addresses. Routers work here.
- **Transport**: connects processes using ports, with TCP (reliable stream) or UDP (fast datagrams).
- **Application**: everything your code speaks: HTTP, DNS, TLS, gRPC, SMTP.

2 Mapping TCP/IP to OSI

The mapping is simple. Link covers OSI layers 1 and 2. Internet is layer 3. Transport is layer 4. Application squashes layers 5, 6 and 7 into one, which is why TLS has no clean OSI number. That is also why engineers still say “L4” and “L7”: **the numbers come from OSI, but the protocols come from TCP/IP.**



“L4” and “L7” keep the OSI numbers, but the protocols are TCP/IP's

Figure 1. TCP/IP folds OSI's top three layers into one application layer and its bottom two into one link layer.

3 Opening shop.example.com, step by step

1. **DNS.** The browser asks a resolver for the server's IP address, usually over UDP port 53. If the answer is cached (browser, OS or resolver), this step costs almost nothing.
2. **TCP handshake** to port 443: SYN, SYN-ACK, ACK. One round trip before any data.
3. **TLS handshake.** The server proves who it is with a certificate signed by a trusted authority, and both sides agree on encryption keys. TLS 1.3 needs one round trip; the older TLS 1.2 needs two.
4. **HTTP.** Only now does the browser send `GET /`, inside the encrypted channel, and the server replies.

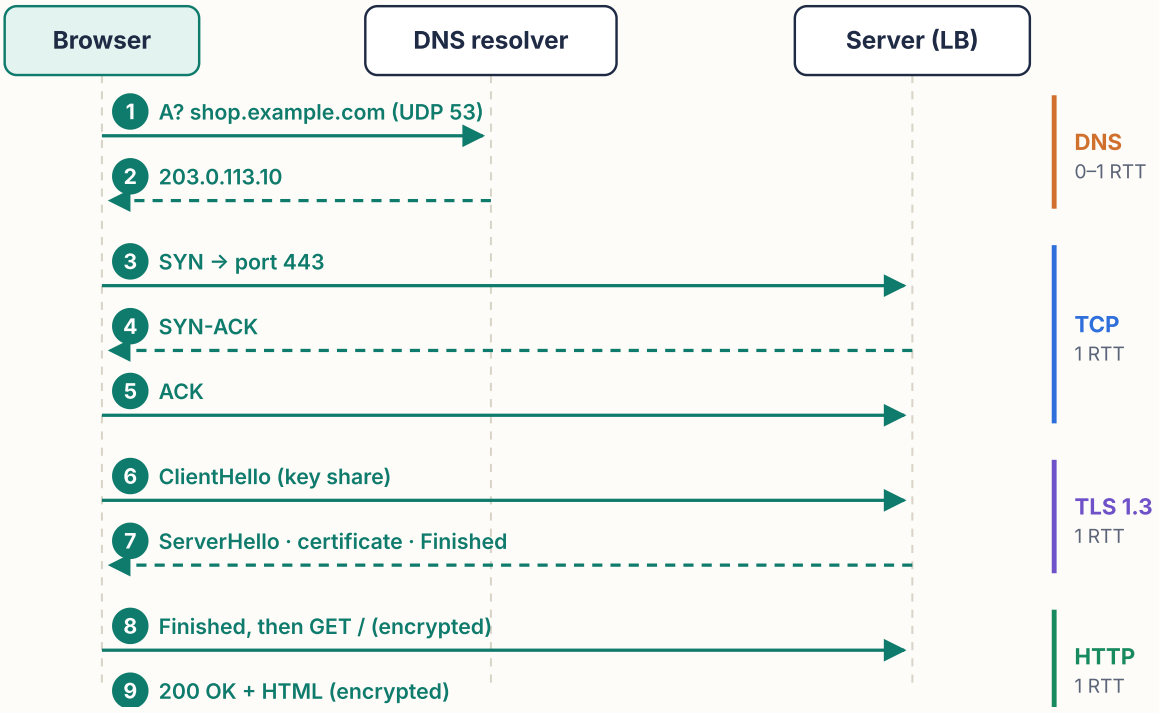


Figure 2. A cold first request: DNS, then TCP, then TLS, and only then HTTP. Solid arrows go to the server, dashed arrows come back.

4 Wrapped, then routed hop by hop

On the way out, the request is wrapped layer by layer: HTTP inside TLS, inside a TCP segment (ports 51514 → 443), inside an IP packet (to 203.0.113.10), inside a Wi-Fi frame. Then it hops: your home router, your ISP, several internet routers, the cloud load balancer, and finally the VM.

At every hop, a router strips the link frame, reads the destination IP, picks the next hop and builds a **new frame** for the next link. The **destination IP stays the same** all the way to the load balancer. (The source IP usually changes once: your home router's NAT swaps your private address for its public one.) The VM unwraps the request in reverse, and the response travels back the same way.

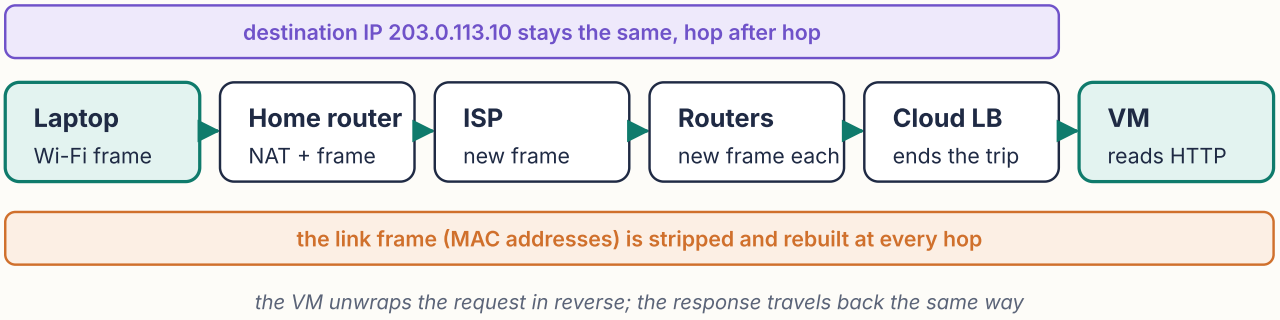


Figure 3. Frames are local to one link and are rebuilt at every hop; the IP packet inside travels end to end.

5 The hidden cost before the first byte

Count the round trips: a DNS lookup, a TCP handshake and a TLS handshake all happen *before a single byte of HTTP*. On a fresh connection, the setup can take longer than the request itself.

WORKED EXAMPLE

A user in Bengaluru opens a site hosted in Frankfurt. The round-trip time (RTT) is about **150 ms**.

- Cold: DNS (say 30 ms to a nearby resolver) + TCP 150 + TLS 1.3 150 + HTTP 150 ≈ **480 ms** to the first response byte.
- With TLS 1.2 instead: one more RTT, ≈ **630 ms**.
- Second request on the same kept-alive connection, DNS cached: just the HTTP round trip, ≈ **150 ms**.

Same server, same code: 3–4× faster just by reusing the connection. A CDN edge 10 ms away shrinks every handshake round trip too.

So the main levers are: **DNS caching** (sensible TTLs), **connection reuse** (HTTP keep-alive, connection pools, HTTP/2), **TLS 1.3 and session resumption**, and **moving the handshake closer** to users with a CDN or edge proxy.

COMMON MISTAKES

- Saying the internet "runs on OSI". It runs on TCP/IP; OSI supplies the layer numbers.
- Thinking the IP address changes at every router. The frame changes; the destination IP does not (until it reaches the load balancer).
- Forgetting DNS and TLS when estimating latency. A cold request costs several round trips, not one.
- Opening a new connection per request between services, paying TCP and TLS every time.

6 Key terms

TCP/IP model: link, internet, transport, application

Handshake: setup messages before data (TCP, TLS)

Certificate: proof of the server's identity, signed by a CA

Resolver: DNS server that looks up names for clients

Hop: one router-to-router step on the path

NAT: rewrites private source IPs to a public one

RTT: round-trip time between two hosts

Keep-alive: reusing a TCP connection for many requests

TTL (DNS): how long an answer may be cached

7 Interview questions

Q1. How does the TCP/IP model map to OSI? BASIC

Link covers OSI 1 and 2, internet is 3, transport is 4, and application covers 5, 6 and 7. That is why TLS has no clean OSI number: TCP/IP just counts it as application.

Q2. What happens when you type a URL and press Enter? BASIC

DNS resolves the name to an IP (usually over UDP). TCP opens a connection to port 443 (SYN, SYN-ACK, ACK). TLS authenticates the server with its certificate and agrees keys. Then the browser sends GET / over the encrypted channel, and IP routes every packet hop by hop to the server and back.

Q3. Which parts of a packet change at each router hop? BASIC

The link-layer frame (source and destination MAC addresses) is rebuilt at every hop, and the IP TTL drops by one. The destination IP stays the same until the packet reaches its destination, such as the load balancer.

Q4. Users far from your only region complain the first page load is slow, though server time is 20 ms. Why, and what helps? SCENARIO

The cold path pays several round trips (DNS, TCP, TLS, HTTP); at 150 ms RTT that is about 0.5 s before any server work matters. Use a CDN or edge proxy to terminate TCP and TLS near users, TLS 1.3, keep-alive and longer DNS TTLs.

Q5. A microservice calls another over HTTPS and opens a new connection per call. p50 is 40 ms with 1 ms of work. What do you change? SCENARIO

Reuse connections: an HTTP client with a keep-alive pool, or HTTP/2 or gRPC multiplexing. That removes the TCP and TLS handshakes from every call, leaving roughly one RTT plus work.

Q6. Where does DNS fit in the layers, and why is it usually UDP? SCENARIO

DNS is an application-layer protocol. A query and its answer usually fit in one small packet each, so UDP avoids a handshake. It falls back to TCP for large answers and zone transfers.

Q7. Why do we still say L4 and L7 if the internet uses TCP/IP? TRICKY

The numbers are a shared vocabulary from OSI. L4 means the transport layer (TCP/UDP, ports) and L7 means the application (HTTP), which map cleanly onto TCP/IP's transport and application layers.

Q8. Is the source IP your server sees the user's laptop address? TRICKY

Usually not. Home and mobile networks use NAT, so the server sees the router's or carrier's public IP. Behind a proxy or L7 load balancer, it sees the proxy's IP, and the client's IP arrives in a header such as X-Forwarded-For.

Next: SD-B-003 · TCP Explained backendroadmap.com